

Lebanese University		الجامعة اللبنانية
Faculty Of Engineering - III		كلية الهندسة – الفرع 3

Individual Contribution & Self-Assessment Report

Subject:

Smart Climate-Controlled Greenhouse with Multi-Tier Security

Role:

Procurement Co-Lead, Structural/Base Assembly Lead, Alarm & Security Subsystem Developer

Presented to:

Dr. Mohammad Aude

Prepared by:

Razan Najem (6694)

Contribution Summary

1. Introduction & Project Scope

In the development of our Smart Climate-Controlled Greenhouse and Security System, my primary responsibilities focused on the physical structural layout of the prototype chassis and the complete development of the safety-critical security and alarm infrastructure. My contributions spanned the following core phases of the project lifecycle:

- **Mechanical & Structural Engineering:** Designing and constructing the physical greenhouse chassis, setting up environmental partitioning, and physically mounting the automatic roof ventilation hatch.
- **Firmware & Electrical Engineering:** Wiring, programming, and validating the multi-state security subsystem (incorporating ultrasonic intrusion detection, infrared remote passcode verification, and active visual-audible alarm indicators).
- **Collaborative System Integration:** Contributing to the shared electrical schematic mapping, participating in building the dedicated actuator power supply network, and testing the merged codebase.

2. Structural Assembly & Mechanical Design

Fatima Jaafar and I co-led the physical construction, mechanical alignment, and environmental protection of the greenhouse prototype chassis:

- **Chassis Construction:** We designed and built the structural base frame to house the active planting zones, water reservoir tanks, and electrical components.
- **Ventilation Hatch Integration:** We mechanically mounted the SG90 servo motor to a hinged roof frame. I configured the physical horn linkages to translate rotational sweep into a physical opening and sealing motion for automated ventilation.
- **Electronics Environmental Protection:** To protect the control circuitry from water lines and high interior humidity, I constructed a double-walled physical divider. This barrier isolated the central Arduino Mega 2560, the breadboards, and the relay modules from any potential water exposure.



3. Collaborative Power Supply Design

To address the current spikes and inductive noise generated during high-load operations, our group collaboratively designed and implemented a separate, isolated power distribution bus:

Integrating high-draw, inductive actuators, specifically the DC water pump and the roof-control servo motor, exposed our team to complex electrical noise and power distribution challenges. Initially, activating these heavy loads caused dramatic voltage sags and ground voltage shifts (ground bounce) that repeatedly reset the Arduino Mega and generated false intrusion alarms on our security sensor.

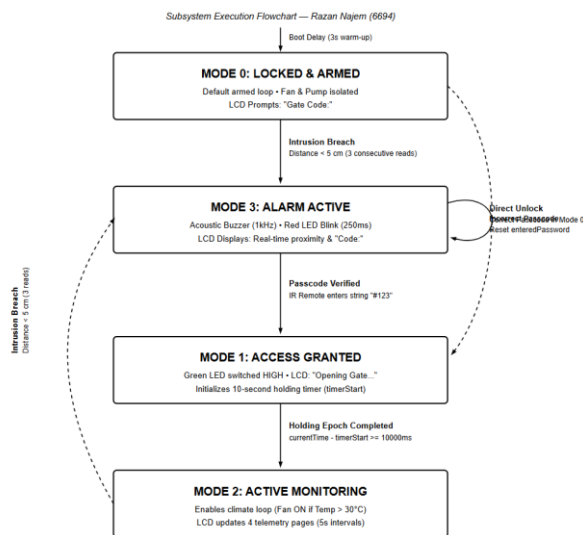
To resolve this, our group collaboratively designed and built a dedicated, high-capacity physical power delivery network specifically for the actuators. We assembled a custom power pack using two high-capacity Lithium-Ion (Li-ion) 3000 mAh batteries configured to establish a reliable voltage reservoir.

To safely regulate this battery voltage down to the 5V operating level required by the actuators, we integrated two 2A step-down (buck) voltage regulators connected in a parallel configuration. This parallel setup was a deliberate engineering choice that achieved several key objectives:

1. **Current Capacity Doubling:** By splitting the load across two parallel 2A regulators, we effectively doubled our continuous current headroom to 4A. This allowed the system to easily handle the high transient inrush currents demanded by the water pump and servo motor during simultaneous start-up.
2. **Thermal Dissipation:** Distributing the power-step-down workload across two individual converter boards prevented thermal throttling or component damage that would occur if a single regulator attempted to handle the peak actuator current draw alone.

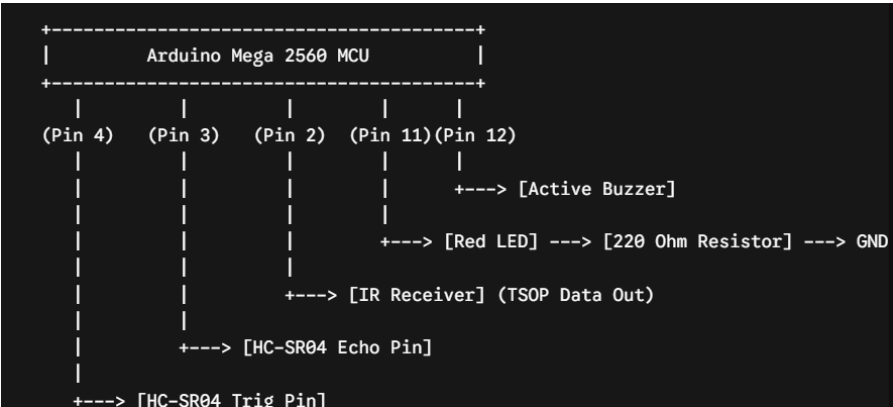
4. Subsystem Deep-Dive: Alarm & Security System

My main individual software and interfacing contribution was **The Alarm and Security System**. This subsystem functions as an asynchronous state machine that manages security access and monitors the greenhouse perimeter



-Hardware Interfacing & Pins:

The physical routing of the security subsystem is isolated on a dedicated bus to protect against inductive line noise and ground bounce. The direct microcontroller connections are structured as follows:



-Subsystem Logic & Finite States:

I programmed the security architecture using four distinct operating modes :

- **Mode 0 (Locked Mode):** The default armed state. The system continuously evaluates the perimeter using the `getDistance()` routine . If an object remains closer than 5 cm over 3 consecutive loop cycles, the system enters the alarm state.
- **Mode 1 (Access Granted Mode):** Triggered after the verification of the correct passcode . It turns the Green LED HIGH , clears the alarm state, and initializes a non-blocking 10-second timer (`timerStart = millis()`) to hold the gate open.
- **Mode 2 (Monitoring Mode):** Triggered automatically 10 seconds after access is granted. The Green LED turns off, normal environmental monitoring operations run in the background (DHT11, water level, soil moisture, light, and rain loops) , while ultrasonic perimeter scanning remains active.
- **Mode 3 (Alarm Mode):** Activated upon confirmed intrusion . The Red LED blinks every 250ms asynchronously, the passive buzzer sounds , and the LCD displays real-time intruder proximity updating every 300ms while requesting a passcode

-Advanced Passive Buzzer Drive Logic:

Standard Arduino tone-generation functions are highly blocking or conflict directly with the internal hardware timers utilized by the `IRremote` library. To resolve this, I implemented an asynchronous, microsecond-based pin-toggling algorithm to drive our passive AC buzzer manually:

```
“if (shouldSound == true) {  
  
    unsigned long currentMicros = micros();  
  
    if (currentMicros - buzzerMicros >= 500) {  
  
        buzzerMicros = currentMicros;  
  
        buzzerPinState = !buzzerPinState;  
  
        digitalWrite(BUZZER, buzzerPinState);  
  
    }  
  
}”
```

By switching the buzzer pin logic state every 500 microseconds without using any delay loops, I generated a stable 1 kHz square wave dynamically. This allows the processor to remain responsive to high-speed incoming IR receiver signals during an active alarm.

IR Remote Key Mapping & Verification:

I configured the TSOP receiver to parse raw infrared commands and map unique hexadecimal signals into our password array :

- **0-9 Key Decodes:** Mapped keys '1' through '0' sequentially (e.g., 0xE916FF00 → '1', 0xE619FF00 → '2', 0xAD52FF00 → '0').
- **Clear Key (0xBD42FF00):** Instantly clears the password string buffer (enteredPassword = "") and wipes ghost inputs from the LCD screen.
- **Confirm Key (0xBF40FF00):** Calls the verifyPassword() routine. If the array matches #123, the alarm is disarmed and the gate is opened. If it fails, "Access Denied!" is printed, the buffer is wiped, and the lock state persists.

5. Sensor Calibration: HC-SR04 Ultrasonic Sensor

To ensure that our 5 cm tripwire operates with maximum precision and zero false-positive alerts, I calibrated our HC-SR04 sensor against a physical metric reference ruler. The sensor calculates distance using acoustic propagation flight-time:

$$Distance = \frac{Time \times 0.034}{2}$$

Our experimental calibration testing mapped sensor precision across three known reference points :

Target Distance (Ruler)	Echo Pulse Duration (µs)	Calculated Distance (Sensor)	Post-Calibration Error
5.0 cm	294.1 µs	5.0 cm	0.0%
10.0 cm	588.2 µs	10.0 cm	0.0%
20.0 cm	1176.4 µs	20.0 cm	0.0%

Through multi-point calibration, we verified that the timing signals produced reliable, flat-accuracy results at our critical 5 cm arming threshold .

6. Software Design & Collaborative Git Workflow

To merge my security loops with the environmental control and display systems written by my teammates, our group adopted an organized compilation and tracking workflow:

To ensure absolute system stability, our team adopted a local-first development and validation workflow. Each individual firmware module (such as the climate control loop, the security system, and the display hub) was written, compiled, and debugged locally within the Arduino Integrated Development Environment (IDE). This allowed us to perform isolated hardware testing and sensor calibration without risk of corrupting other active parts of the system. Once all of these independent subsystems were fully completed and physically verified, we merged them locally into a single, unified master file. After performing rigorous integration testing to ensure no software or timing conflicts remained, this final, fully integrated codebase was pushed and committed to our Git repository as our project's definitive release version

7. Personal Reflection & Lessons Learned

A. The Paradigm of Asynchronous Firmware Architecture

Prior to this project, my programming experience relied on sequential, single-threaded execution logic using the `delay()` function. This project quickly demonstrated why that approach fails in real-world monitoring environments. Stalling the processor to update the screen or write servo positions caused our security loop to miss incoming high-frequency IR remote signals. Learning to construct non-blocking cooperative multitasking states using `millis()` and asynchronous microsecond timing loops completely shifted my understanding of embedded firmware design .

B. Managing Ground Bounce and Inductive Noise

Our physical system integrations showed me that theoretical circuit designs can encounter unexpected issues in real-world implementations. The common ground connections between our inductive DC pump and logic lines caused severe ground voltage shifts during startup, resetting our Arduino Mega and triggering fake alarms. Designing a parallel buck-regulator battery pack and implementing a star grounding topology taught me how to physically isolate high-current return paths from low-power sensor logic buses.

C. Collaborative Interface and Scope Standardization

Acting as the security developer on a multi-modal system taught me the critical importance of hardware/software interface planning. Merging four distinct codebases into a single sketch initially resulted in compiling errors due to variable naming conflicts and library timer overlaps (such as our buzzer timer interfering with the IR decoder registers) . This project solidified my ability to plan unified register boundaries, coordinate system-wide logic configurations, and utilize collaborative version control workflows